Yatranook

AI-Driven Intelligent Tourism Assistant for Odisha

Explore Odisha Smarter with Trusted AI

Contents and Executive Abstract

1. Introduction and background | 2. Problem statement and existing approach
3. Objectives, proposed solution and users | 4. Functional and non-functional requirements
5. System architecture and end-to-end workflow | 6. AI methodology, RAG and knowledge management
7. Personalized itinerary planning and scenarios | 8. External data, verification, safety and privacy
9. Technology stack and implementation roadmap | 10. Testing, evaluation, risks and expected outcomes
11. Conclusion and references |

Executive abstract

Yatranook is a proposed AI-powered tourism assistant designed to help visitors discover destinations, understand cultural heritage, plan practical trips and adapt itineraries when conditions change. The concept combines a conversational interface with a curated tourism knowledge base, source-aware responses, personalized itinerary generation, map-aware planning and selected live information services where reliable providers are available.

The distinguishing approach is to treat tourism assistance as a decision workflow rather than a standalone chatbot. User preferences and trip constraints are collected; relevant destination records and tourism documents are retrieved; a language model drafts a response using that context; a planning layer checks time, budget and route feasibility where reliable data exists; and the interface discloses sources, freshness and uncertainty. Retrieval-Augmented Generation (RAG) can reduce unsupported answers but cannot guarantee factual correctness.

The proposed initial implementation is a responsive web application using React and Tailwind CSS, a Python/FastAPI backend, PostgreSQL with pgvector, an LLM and embeddings, and optional mapping/weather APIs. The project should be evaluated on a reviewed factual-answer set, source correctness, unsupported-claim rate, itinerary constraint satisfaction, cost/time estimate error, API failure handling, latency, multilingual quality and actual user feedback. No impact, performance or implementation result is claimed before testing.

Document scope and evidence status

This document describes the proposed system, not a completed product. The traveler pain points are hypotheses requiring user research. External integrations are optional until access, terms and reliability are confirmed. Any demonstration itinerary or budget below is illustrative and must not be presented as a live quote or guarantee.

1. Introduction and Background

Odisha offers varied travel experiences spanning temple heritage, archaeological monuments, beaches, lakes, waterfalls, wildlife, festivals, handicrafts, cuisine and local cultural traditions. The official Odisha Tourism portal organizes destinations and experiences across the state, while UNESCO documents the heritage significance of the Konark Sun Temple. These public resources provide starting points for a curated destination knowledge base, subject to source review, attribution and freshness checks.

A traveler's decision is multi-dimensional. They may need to choose places based on interests, find enough time to visit them, estimate transport and activity costs, understand cultural context, check whether information is current and adjust when the plan changes. These tasks can span multiple information providers, and the traveler may have to reconcile facts and estimates manually.

2. Problem Statement

Problem statement: tourists may face fragmented, unevenly structured or difficult-to-compare travel information when discovering destinations and assembling practical itineraries for Odisha. A proposed assistant should reduce the effort of combining information while making evidence limits visible. The problem is to be validated; this document does not claim that all tourists experience every pain point or that the severity is already quantified.

Pain point	Traveler difficulty	Possible consequence	Proposed response	How to measure
Destination discovery	Hard to match places to interests and location	Time spent searching; overlooked alternatives	Rank destinations with reasons and source-backed details	Relevance ratings; task completion time
Trip feasibility	Budget, time, distance and visit duration must be reconciled	Overpacked or unrealistic schedules	Apply constraints and show estimates/assumptions	Constraint satisfaction rate
Cultural context	Listings may provide limited explanations or etiquette	Lower understanding or inappropriate expectations	Retrieve source-backed cultural/heritage explanations	Reviewed factual accuracy; usefulness rating
Language access	Information may not be easy to use in preferred language	Misunderstanding or extra translation effort	Prioritize English/Odia/Hindi and evaluate quality	Human-reviewed language quality
Information freshness	Hours, weather and transport details can change	Uncertainty and possible wasted visits	Show source, timestamp and freshness state	Freshness compliance; stale-data incidents
Plan changes	Closures, rain or delays invalidate the original schedule	Extra planning effort during a trip	Revise while preserving remaining constraints	Successful adaptive replanning rate

Research needed: interview domestic and international visitors, families, solo travelers and heritage-focused visitors; observe a real planning task; ask which sources they trust; measure current effort; and test prototypes before making quantified claims.

3. Existing Approach and Identified Limitations

A common alternative is to use separate destination websites, maps, blogs, social posts, travel listings and general-purpose chatbots. These sources can be useful, but the traveler may still need to combine them and judge whether a claim is current or supported. This is a design analysis, not a claim that every existing service is inadequate.

Approach	Potential limitation	Yatranook design response
Search across sites	Different formats, detail levels and update cycles	Curated destination records with source and verification metadata
General chatbot	May produce fluent answers without cited, current evidence	Retrieve relevant records and display sources/uncertainty
Static itinerary list	May ignore time, distance, budget or opening constraints	Constraint-aware draft plus feasibility checks
Separate maps/weather	Information is not always connected to itinerary decisions	Use selected provider responses when available; label estimates
Manual re-planning	User repeats the search when a trip changes	Preserve trip context and revise affected stops

4. Project Objectives

- Build a searchable, curated dataset of Odisha destinations with provenance and data-quality fields.
- Provide conversational tourism answers grounded in retrieved records and documents.
- Generate personalized, day-wise itinerary drafts from location, dates, budget, duration and interests.
- Use route, weather and opening-hour integrations only when appropriate, available and sufficiently reliable.
- Adapt itineraries when constraints change while exposing trade-offs and unknowns.
- Evaluate factuality, evidence attribution, planning feasibility, resilience and user experience before claiming benefits.

5. Proposed Solution

Yatranook combines six cooperating capabilities: conversational assistance; destination discovery; an AI cultural and heritage guide; constraint-aware itinerary generation; map-aware planning; and trust/freshness controls. The traveler journey is Discover → Understand → Plan → Navigate → Adapt → Explore with Confidence. The proposed system retains trip context, asks clarification questions when essential inputs are missing and presents the itinerary as a recommendation rather than a guarantee.

Target users and use cases

Primary research segments include domestic tourists planning short breaks; international visitors who may need multilingual context; families with pacing needs; solo travelers looking for flexible options; and heritage enthusiasts who want deeper explanations. The first segment to prioritize should be selected from interviews and observed planning behavior, not assumed in advance.

6. Functional Requirements

ID	Requirement	Acceptance intent
FR-01	Destination search	Search and filter destinations by region/district, interests and available metadata.
FR-02	Conversational Q&A	Accept natural-language questions and maintain relevant session/trip context.
FR-03	Clarification	Ask for essential missing values such as dates, starting point, pace or budget.
FR-04	Evidence retrieval	Retrieve relevant curated records and document snippets before generating factual answers.
FR-05	Personalized itinerary	Generate a day-wise draft with activities, visit durations, suggested sequence and alternatives.
FR-06	Feasibility checks	Check stated budget/duration and reliable access/hour constraints where data exists.
FR-07	Source transparency	Display source title/link and last verified or retrieved time where available.
FR-08	Selected live data	Fetch weather/routes/visitor updates only through configured reliable providers.
FR-09	Adaptive revision	Update stops while preserving the user's remaining constraints.
FR-10	Language options	Prioritize English, Odia and Hindi when the selected model/translation path supports them.

7. Non-Functional Requirements

Quality attribute	Requirement / verification
Correctness	Use reviewed test sets; track source-supported claims and unsupported-claim rate.
Transparency	Make the difference between live data, reviewed records and AI estimates visible.
Resilience	Handle empty results, timeouts, provider errors and partial data without fabricating results.
Performance	Measure median and tail response latency under representative test load.
Security and privacy	Validate inputs, minimize stored personal data, protect secrets and define retention.
Maintainability	Separate routes, services, retrieval, planning and provider adapters; test each boundary.
Accessibility and usability	Test keyboard access, responsive layouts, readable contrast and comprehension.
Localisation quality	Review translation, place names, cultural nuance and language-specific edge cases.

Acceptance thresholds should be set before testing. Do not backfill a target after seeing results or report metrics without the test-set size, date, methodology and limitations.

8. System Architecture

The architecture is a proposed layered design. The interface sends a request to the FastAPI application layer; request validation and session/trip context feed search and itinerary services; the AI layer retrieves evidence and assembles context; the planning layer checks known constraints; and the data layer stores destination information, document chunks and source metadata. Optional providers supply route, weather or other current information where configured.

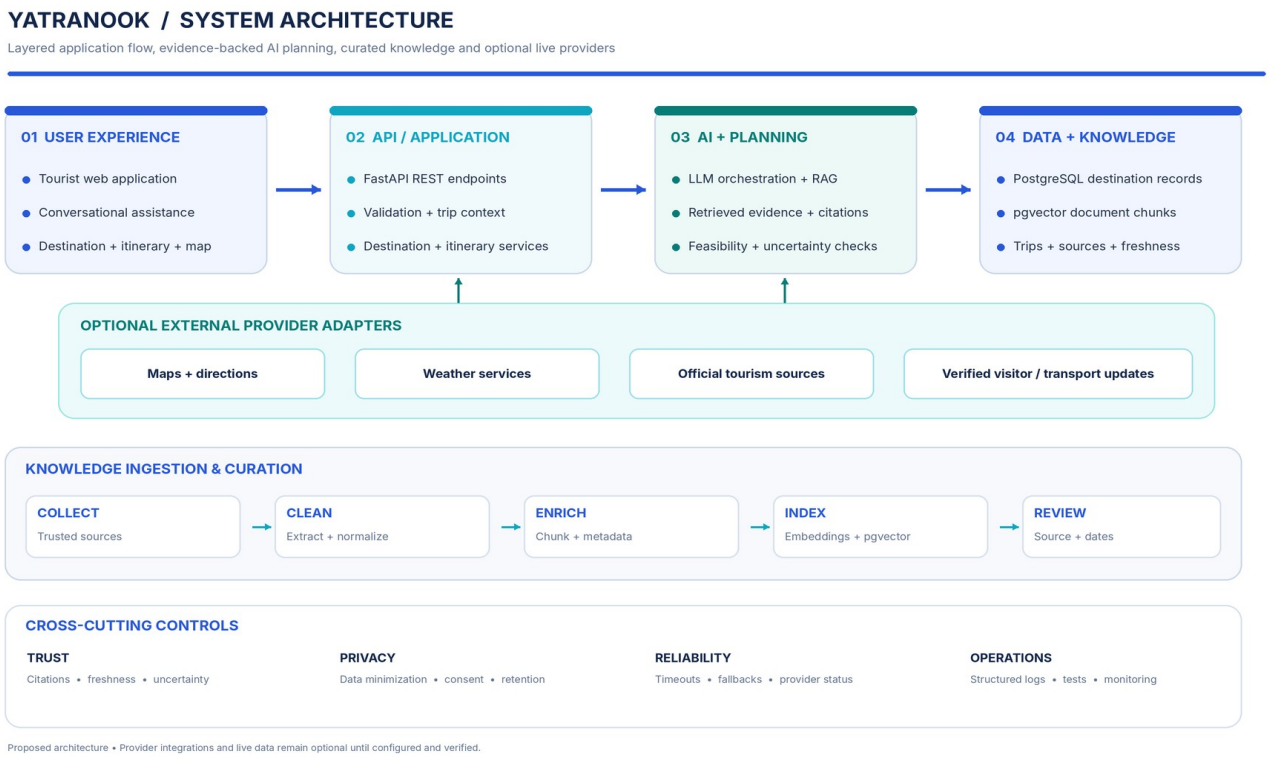


Figure 1. Proposed layered system architecture. External providers are optional integrations, not asserted as deployed.

Component responsibilities

Component	Responsibility
Web interface	Collect preferences; show chat, itinerary, destination detail, map and evidence labels.
FastAPI layer	Validate requests; route to application services; manage errors and structured logs.
RAG service	Retrieve relevant content; assemble bounded context; preserve source references.
Planning service	Rank destinations; estimate visit/travel time; apply constraints; validate the draft.
PostgreSQL / pgvector	Store structured destinations, source metadata, itinerary preferences and embedding-backed chunks.
Provider adapters	Encapsulate map, weather, tourism and verified visitor/transport sources; return freshness and failure states.

9. End-to-End Workflow

The workflow should stop and clarify when required inputs are missing, disclose when reliable information cannot be found, and revise infeasible plans rather than quietly presenting them as valid.

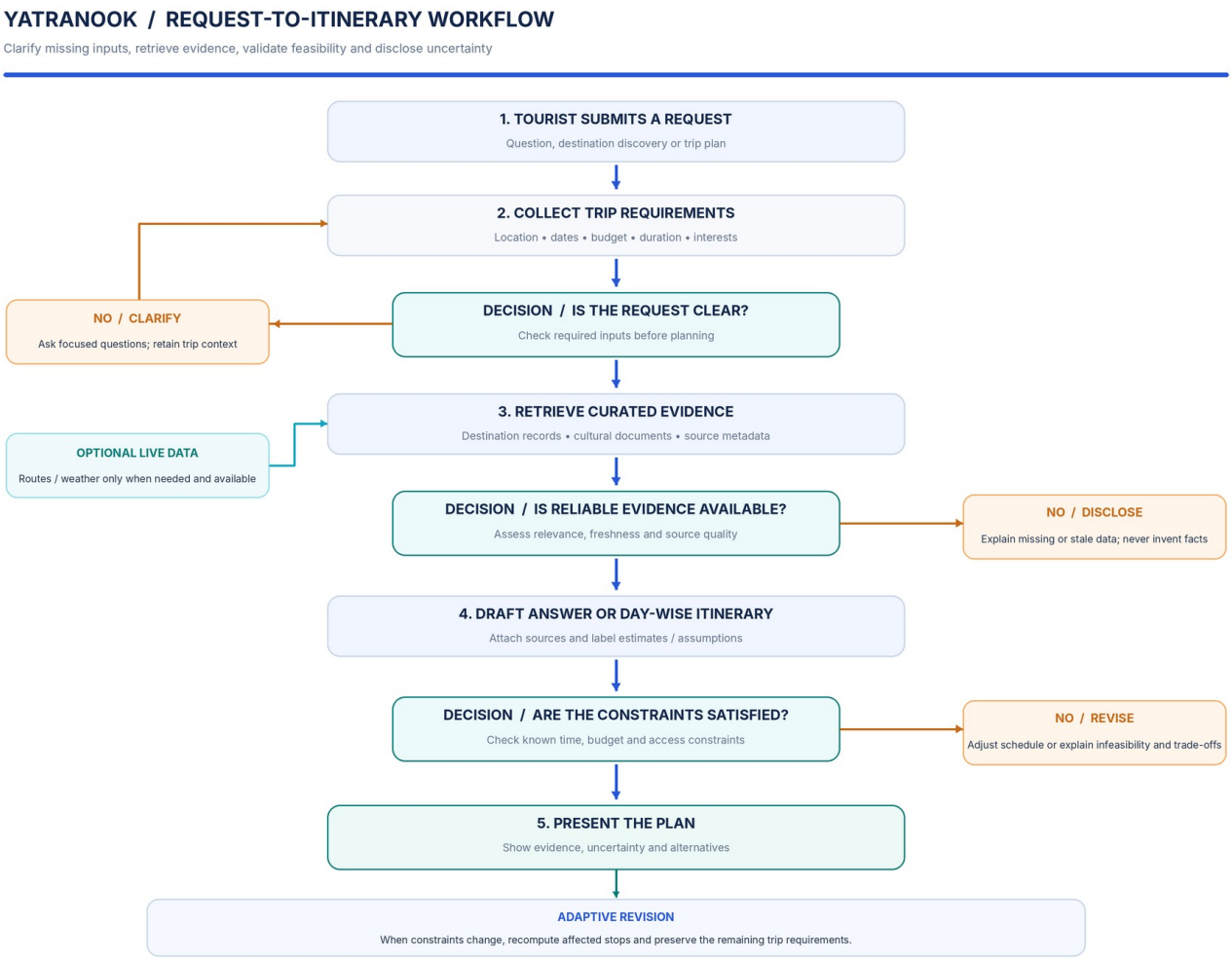


Figure 2. Proposed request-to-itinerary workflow with evidence and feasibility decision points.

Workflow logic

1. Collect trip requirements: starting location, destination/region, date range, budget, duration, interests and travel constraints.
2. Check whether the request is clear; ask focused clarification questions rather than assuming missing facts.
3. Retrieve the most relevant destination records, cultural documents and source metadata.
4. Fetch live provider data only when required and available; preserve retrieval time and provider status.
5. Draft recommendations or a day-wise itinerary with source links and labelled estimates.
6. Validate known constraints. If infeasible, revise the route or explain the trade-off and offer alternatives.
7. Present the plan. When the traveler changes a constraint, update the affected schedule while preserving the rest.

10. AI Methodology: Retrieval-Augmented Generation

RAG adds a retrieval stage before language generation. It can improve grounding by supplying relevant material to the model, but it does not prove that the retrieved content is correct, complete or current. Source selection, preprocessing, metadata, retrieval evaluation and response checks remain necessary.

Stage	Method
1. Collect	Use official or otherwise trusted tourism sources and record each source's identity, access terms and retrieval date.
2. Clean and segment	Extract text, normalize formatting, split into coherent chunks and retain links to the parent source.
3. Enrich metadata	Attach destination/district, topic, language, source type, last-verified date and validity window where relevant.
4. Index	Create embeddings for semantic retrieval and store chunks/metadata in PostgreSQL with pgvector, alongside structured destination records.
5. Retrieve	Use semantic similarity with metadata filters; retrieve only relevant context for the current question.
6. Generate and verify	Ask the LLM to answer using the supplied context, avoid unsupported claims and return source references.
7. Disclose uncertainty	When evidence is absent, stale, inconsistent or too weak, say so and avoid inventing hours, prices or official guidance.

11. Destination Data Model and Knowledge Management

A destination record can include a stable ID, name, aliases, district/region, coordinates, categories, descriptive text, access information, estimated visit duration, accessibility notes, source references, last-verified date, validity status and language. Each field should carry provenance when it matters. Unknown values should remain unknown rather than being filled with plausible guesses.

Suggested related entities include Destination, Source, DocumentChunk, Itinerary, ItineraryStop, UserPreference and ProviderObservation. Store current observations separately from relatively stable cultural descriptions. Preserve which source supported which field so corrections can be traced, and provide a maintenance workflow for stale or disputed records.

Information freshness tiers

Tier	Examples	Handling
Static / slow-changing	Architecture, historical context	Review source and updates periodically; still cite source.
Periodically reviewed	Destination descriptions, facilities	Show last-reviewed date and schedule refresh.
Live response	Weather or route estimate	Show provider and retrieval time; handle timeout / missing data.
Unverified / unavailable	Unknown hours, transport, closure or emergency data	Clearly disclose unknown status; do not invent a replacement fact.

12. Personalized Itinerary Planning Algorithm

Start with a transparent weighted scoring model and deterministic constraints. A more advanced route optimization algorithm should be introduced only if realistic test cases show that the simpler approach is insufficient.

8. Input parsing: collect budget, trip duration, date, start/end location, interests, preferred pace, party context and must-visit places.
9. Candidate retrieval: select destinations matching interests and the trip region; exclude records with disqualifying known constraints.
10. Candidate scoring: score interest match, location fit, diversity, destination quality of evidence and detour cost.
11. Schedule construction: order candidate stops using route estimates where available, visit duration and practical time windows.
12. Constraint validation: compare estimated total time and spending with user limits; flag unknown costs, unavailable hours and weak route data.
13. Output: present a draft schedule with evidence, assumptions, travel-time/cost estimates, alternatives and limitations.
14. Adaptive revision: when the user changes time, budget or a stop, recompute the affected schedule and preserve constraints not changed.

Illustrative scoring model

Score(destination) = w1 × interest_match + w2 × location_fit + w3 × category_diversity + w4 × evidence_quality − w5 × detour_penalty. Weights are tunable parameters, not proven values; calibrate them through held-out scenarios and user relevance ratings.

Hard constraints may include total planned time ≤ available duration, estimated cost ≤ user budget, exclusion of known closed locations, and non-overlapping visit windows. Unknown values must not be treated as confirmed availability. Route estimates depend on the provider, selected travel mode, time of travel and returned data.

13. Demonstration Scenarios

Scenario	Example request	Expected system behaviour
A. Budget trip	"I have ₹3,000 and two days in Puri. Plan temples, beaches and local food."	Capture trip dates/group size and whether arrival transport is included; produce a draft with labeled cost ranges and verified sources. The example allocation below is not a live quote.
B. Nearby discovery	"I am near Bhubaneswar. Suggest nature destinations for a day trip."	Use current user-provided location or explicit starting point, destination metadata and route provider if available; explain fit and driving uncertainty.
C. Adaptive support	"My planned attraction is closed, and I have only three hours remaining."	Check a reliable status source if available; remove the unavailable stop; recompute timing, distance and cost; disclose when closure status cannot be verified.

Illustrative ₹3,000 allocation for a two-day Puri scenario: stay ₹900, food ₹600, local travel ₹700, activities ₹400 and buffer ₹400. Total = ₹3,000. These numbers are invented solely to demonstrate a budget constraint and are not verified prices. Actual feasibility depends on group size, season, dates, transport to/from Puri, accommodation and current quotations.

14. External API and Real-Time Data Strategy

Integrations should be added behind provider adapters so the core application can still work when a provider is unavailable. No specific vendor is selected in this proposal. Provider selection should consider coverage, pricing, licensing/terms, attribution rules, rate limits, data freshness, privacy implications and failure behaviour.

Provider class	Potential data	Requirements
Maps / directions	Coordinates, distance, travel-time and	Label provider, mode and retrieval time; avoid calling a

Provider class	Potential data	Requirements
	route geometry	route live navigation unless it is.
Weather	Current conditions or forecast for travel dates	Show forecast period and retrieval timestamp; gracefully handle missing results.
Official tourism sources	Destination descriptions, events, visitor guidance	Respect source terms; track URLs, retrieval date and verification state.
Transport / visitor status	Schedules, hours, closures or updates if an authoritative provider exists	Display only supported fields; do not invent emergency, transport or opening details.
Translation / speech	Multilingual text or optional voice input/output	Test place names, cultural terms, dialect and error handling with human review.

15. Verification, Safety and Privacy

- Source attribution: display source title and link; show last-verified date or retrieval time where available.
- Claim boundaries: use retrieved evidence for factual statements; distinguish exact source facts from AI-generated synthesis and estimates.
- Emergency safeguards: use only verified official emergency/visitor-assistance details. If verification is unavailable, say so and direct the user toward official channels without inventing numbers.
- Freshness handling: use different update policies for cultural history, reviewed destination data and live weather/routes.
- Privacy: minimize collected personal information; communicate why location and travel details are requested; protect secrets; set access and retention policies.
- Security: validate input, protect API credentials, rate-limit expensive routes where needed, log errors without needlessly logging sensitive trip data, and test authorization boundaries if accounts are added.

Known limitations

RAG may retrieve the wrong or stale passage; external APIs may be unavailable or costly; destination records may be incomplete; estimated costs vary by season and traveler; route feasibility depends on transport mode and conditions; multilingual responses can mistranslate culturally important content; and tourist preferences are subjective. The product should show these limits instead of hiding them.

16. Testing and Evaluation Strategy

Testing should progress from isolated components to end-to-end traveler tasks. The evaluation dataset should include normal questions, ambiguous questions, conflicting sources, stale records, missing information, language variation, infeasible budgets, API failures and mid-trip changes.

Test layer	What it covers	Evaluation method
Unit tests	Parsing, scoring, constraint functions, metadata handling and fallback logic	Deterministic fixtures; boundary and invalid-input cases.
Retrieval tests	Whether relevant source passages are retrieved for reviewed questions	Annotated question-to-source set; precision/recall or rank-based review.
Factuality tests	Correctness and evidence support of generated responses	Human-reviewed claims; unsupported-claim and citation correctness rates.
Planning tests	Budget, timing, ordering, known opening constraints and alternatives	Scenario matrix; verify hard constraints and identify honest infeasibility.
Integration tests	Database, provider adapters, API schemas and errors	Mocks for provider outages; controlled integration runs where access exists.
End-to-end tests	Realistic trip planning and adaptive changes	Representative tasks, instrumentation and user observation.
Language/usability tests	English, Odia and Hindi quality; interface clarity and accessibility	Native/competent reviewer feedback, task success and satisfaction.

17. Expected Outcomes and Potential Impact

If the proposed system performs well in validation, it may reduce the effort required to compare destinations, make itinerary assumptions easier to inspect, improve access to cultural context and support faster replanning. These are intended outcomes, not proven impacts. User studies should compare the prototype with participants' current planning process and capture both success and failure cases.

18. Risks, Constraints and Mitigation

Risk	Mitigation approach
Incorrect or stale facts	Maintain source provenance, review schedules, visible freshness and user reporting/correction flows.
LLM hallucination	Use bounded retrieval context, response instructions, factuality tests and explicit uncertainty fallbacks.
Infeasible itinerary	Keep deterministic constraints outside the LLM; recalculate schedule and expose trade-offs.
API outages / costs	Use provider adapters, timeouts, quotas, caching where permitted and clear unavailable-data states.
Biased coverage	Audit destination coverage across districts/categories and collect user feedback on missing places.
Language/cultural error	Test with reviewers familiar with the language and cultural context; label limits and correct records.
Privacy / secret exposure	Minimize stored data, protect secrets, define retention, redact logs and review access controls.
Scope overload	Launch a small curated dataset and core workflow before adding voice, broad live integrations or advanced optimization.

19. Future Enhancements

Potential extensions include voice input/output, accessibility-aware itineraries, festival-specific planning, verified local experience listings, saved trips, collaborative planning, richer district-level discovery and carefully bounded on-trip alerts. Each extension should follow evidence that users need it and that required data can be maintained responsibly.

20. Conclusion

Yatranook proposes a practical AI tourism assistant that connects destination discovery, cultural explanation and itinerary planning through a source-aware workflow. Its core differentiator is not simply conversation: it is the combination of retrieval, constraints, freshness metadata, explicit uncertainty and adaptive replanning. The next milestone is research and a small demonstrable prototype, followed by evaluation. Credible claims about accuracy, feasibility or user value must be earned through testing.

21. References and Source Notes

The sources below support the contextual destination references in this proposal. They do not validate the user pain points, prove product-market fit or establish the status of any proposed implementation. Record the exact page and retrieval date when ingesting material into the product knowledge base.

1. Odisha Tourism - Official portal

https://odishatourism.gov.in/en/home

Overview of Odisha travel experiences and destination discovery.

2. Odisha Tourism - Tourist destinations

https://odishatourism.gov.in/en/tourist-destinations

Official destination listing and regional/district discovery.

3. Odisha Tourism - Puri

https://odishatourism.gov.in/en/destinations/puri

Official destination context, named attractions and visitor information for Puri.

4. UNESCO World Heritage Centre - Sun Temple, Konârak

https://whc.unesco.org/en/list/246/

Heritage description and World Heritage listing for the Konark Sun Temple.

5. Government of Odisha, Department of Tourism - Tourism Policy

https://dot.odisha.gov.in/en/notifications/tourism-policy

Official tourism-policy resources; consult relevant current documents when defining public-sector context.